CS 426 Examination

Large-Scale System Design and Architecture – Answer Key

1. (B) Write-through caching.
In write-through caching, every write operation updates both cache and database synchronously. This ensures cache and database are always consistent, though it adds latency to write operations. Write-behind is faster but risks data loss; cache-aside can have stale data.

2. (C) Any two of the three properties, but not all three.
The CAP theorem states that in the presence of network partitions (P), you must choose between consistency (C) and availability (A). Most systems choose CP (consistent but may be unavailable) or AP (available but eventually consistent). CA is theoretically possible but unrealistic since network partitions are inevitable.

3. (B) To distribute incoming requests across multiple servers for scalability and reliability.
Load balancers distribute traffic using algorithms like round-robin, least connections, or weighted distribution. This prevents any single server from being overwhelmed and provides fault tolerance—if one server fails, traffic routes to healthy servers.

4. (B) Hash-based sharding.
Hash-based sharding applies a hash function to the shard key (e.g., user ID) to determine which shard stores the data: shard = hash(key) mod N. This distributes data relatively evenly but makes range queries difficult.

5. *Sample answer:*
Eventual consistency: After an update, all replicas will eventually converge to the same value, but they may temporarily differ. Reads might return stale data.

Trade-offs:

- **Strong consistency**: All reads see the latest write. Requires coordination (locks, consensus), reducing availability and increasing latency
- **Eventual consistency**: Higher availability and performance, but complexity in handling conflicts and stale reads

Choose eventual consistency when:

- Availability is critical (e.g., social media feeds, shopping carts)
- Temporary staleness is acceptable
- System must tolerate network partitions
- Examples: DynamoDB, Cassandra, DNS

6. *Sample answer:*
Microservices: Architecture where application is composed of small, independently deployable services, each responsible for a specific business capability.

Advantages over monolith:

- Independent scaling of services based on demand
- Technology diversity (each service can use different languages/frameworks)

- Faster deployment cycles (teams can release independently)
- Fault isolation (failure in one service doesn't crash entire system)

Additional complexities:

- **Deployment**: Need orchestration (Kubernetes), service discovery, configuration management
- **Debugging**: Distributed tracing required, harder to track requests across services
- **Communication**: Network calls are slower and less reliable than in-process calls; need to handle failures, timeouts, retries
- **Data consistency**: Managing transactions across services is challenging

7. *Sample answer:*
Distributed message queue: Intermediary that decouples producers (who send messages) from consumers (who process them), storing messages until consumed.

Purpose and benefits:

- **Asynchronous processing**: Producers don't wait for consumers, improving responsiveness
- **Load leveling**: Queue absorbs traffic spikes; consumers process at their own pace
- **Reliability**: Messages persist in queue even if consumers fail
- **Scalability**: Multiple consumers can process messages in parallel

Implementation (Kafka example):

- Messages organized into topics (categories)
- Topics divided into partitions for parallelism
- Consumer groups allow multiple consumers to process messages
- Provides ordering guarantees within partitions

Patterns enabled:

- Event-driven architecture, pub-sub messaging, task queues, stream processing

8. *Sample answer:*
Token bucket algorithm:

- Bucket holds tokens; tokens added at constant rate
- Each request consumes one token; rejected if bucket empty
- Allows bursts up to bucket capacity
- Good for APIs with occasional traffic spikes

Leaky bucket algorithm:

- Requests enter bucket; processed at constant rate ("leak")
- If bucket full, new requests rejected
- Smooths traffic, prevents bursts

- Better for strict rate enforcement

Distributed implementation:

- **Centralized counter**: Use Redis with atomic operations (INCR, EXPIRE) to track request counts per user/IP. Fast but single point of failure
- **Distributed coordination**: Use consistent hashing to route user requests to specific rate-limiting servers; reduces contention
- **Approximate counting**: Each server tracks counts locally; periodically sync. Faster but less accurate
- **Sliding window**: Use sorted sets in Redis to track timestamps of recent requests

Most production systems use Redis-based token bucket with sliding windows for accuracy and performance.